

CarePulse — Software Testing Report

SE3090 Assignment 2: Software Testing and Quality Evaluation

Report date: 3 October 2026. Testing dates: 1–3 October 2026.

Status: prepared for team review. Role mapping, individual testing evidence and the required AI declaration still need confirmation before submission.

Team details

Student ID	Name
IT24101521	Bandara EMGO
IT24100966	Kaluarachchige P.E.K
IT24100009	Walpala M.W.H.T
IT24101037	Savindu

The user confirmed that no member has yet completed an individual testing task. The recorded work was prepared and executed with Codex assistance. Tests cover all four project areas, but each member must personally verify and demonstrate a meaningful test. Mapping these names to Students 1–4 is still pending.

1. Project and purpose

CarePulse connects patient records, AI symptom triage, doctor appointments and nurse dispatch. It uses a .NET API, PostgreSQL, a React staff website and a Flutter patient/nurse app.

We tested whether these parts save the correct data, enforce access rules and work together. We also tested response time, security, recovery and selected form accessibility checks. This report describes software behavior, not the medical accuracy or clinical approval of the system.

2. Test plan

The main risks were unauthorized access to patient data, duplicate bookings or nurse assignments, actions without doctor approval, and failures being shown as success. Tests used normal inputs, invalid inputs, exact limits and simulated failures.

Area	Tool and purpose
Backend and database	xUnit, Moq and WebApplicationFactory with isolated PostgreSQL: check API rules, saved data, migrations and concurrent requests.
React website	Jest and React Testing Library: check forms, protected routes, state and errors.
Flutter app	flutter_test: check validation, widgets, sessions and API responses.
Integrated workflow	A local HTTP harness with actual React and Flutter service code: check the same case across clients.
AI	Deterministic xUnit tests and a small live Gemini sample: check output format, allowed actions, approval and failures.

Area	Tool and purpose
Performance	A Python HTTP script: measure valid responses, errors and response times at different client counts.
Security	Automated HTTP tests: attempt unauthorized access, invalid uploads and invalid requests.
Reliability and accessibility	Recovery/session tests and axe-core: check failure handling and selected form markup.

The Python performance script records every request and checks response content. The security tests send requests to the actual API middleware and verify that rejected writes save nothing. These are suitable tools for the selected local scope; no ZAP scan was performed.

Testing used synthetic data on a local Mac, .NET 8.0.423, PostgreSQL 18.4 and Flutter 3.47.1. Baseline web checks used Node 22/npm 10; the later accessibility run used Node 24.11.1/npm 11.6.2. Automated tests did not use the shared Neon database. AI was replaced with controlled responses except in the clearly identified live planner sample.

Work started from commit 1c638e6. Later tests and the prompt fix were uncommitted during these runs; evidence metadata records source hashes or patches. This report does not claim a fresh CI pass for the final submission commit.

The work followed this order: plan and baseline; missing edge cases; client integration; AI; performance; security; reliability/accessibility; report. Proposed responsibilities are Student 1: records/security, Student 2: triage/AI, Student 3: scheduling/performance, Student 4: dispatch/recovery. These are proposed areas, not proof of personal contribution. See TEST_PLAN.md for the full plan.

3. Results

Each row below is a separate run. Some runs repeat the same tests, so the rows must not be added together as a unique total.

Run	Result
Original baseline	116 backend, 33 React and 39 Flutter tests passed: 188 total, no failures or skips. Web build passed.
Backend edge-case expansion	128 passed: 121 API tests and 7 safety tests. No failures or skips.
Cross-client service workflow	All 4 ordered phases passed.
AI deterministic checks	40 passed before and 40 after the prompt fix; these are the same cases rerun.
Live AI planner	3 initial responses passed format checks, but review found unsupported actions. All 3 corrected samples passed the bounded retest.
Performance	2,400 measured reads returned valid data with zero errors.
Security	17 passed: 11 new probes and 6 existing workflow/security cases.

Run	Result
Latest web regression	37 passed, including 4 accessibility checks; 1 optional HTTP-harness test skipped because its session setup was absent.
Focused reliability	2 backend and 3 Flutter tests passed.

The original API suite measured 90.09% line coverage and 49.02% branch coverage for its collected backend scope. This is a baseline measurement, not whole-system coverage or proof that every behavior is correct. Separate safety-suite coverage is not added to it.

The case document, TEST_CASES.csv, groups related scenarios. It is separate from framework test counts. Original reports, commands and run details are listed in EVIDENCE_INDEX.md.

4. Integrated workflow

The automated workflow followed one synthetic case:

1. Flutter service submitted the patient's symptoms.
2. React service approved the case and assigned a nurse.
3. Flutter nurse service sent location, confirmed arrival and completed the visit with vitals. The patient service checked completion.
4. React refreshed and confirmed completion and nurse availability.

The test used real HTTP, production client service/store code and isolated PostgreSQL. AI responses and secure storage were substituted. All four phases passed. This establishes a cross-component workflow; it does not establish that every browser or phone screen works. Full UI acceptance case INT-02 remains Not run in the QM evidence set.

5. Performance and security

The performance dataset contained 501 patients, 51 doctors, 1,001 slots and one nurse. Requests read the doctor directory, cardiology slots and a patient page. Each stage sent 600 requests after warm-up.

Concurrent clients	p95 response time	Errors
1 — baseline	4.591 ms	0
5 — target load	9.921 ms	0
20 — exploratory load	30.684 ms	0
1 — final sample	2.052 ms	0

p95 means 95% of measured requests completed within that time. The planned five-client target was below 1,000 ms with fewer than 1% errors. It passed. These were short local read tests, not long-duration tests or proof of production capacity. They excluded live AI and concurrent writes.

Security probes checked anonymous access, expired or changed tokens, role restrictions, cross-patient access, invalid uploads and invalid coordinates. All 17 selected tests passed. Rejected writes were checked for unwanted database changes. No new vulnerability was found in these tests. Deployment security, dependency scanning and a full penetration test were not covered.

6. Defect and retest

QM-AI-001: the planner suggested actions the system cannot perform.

The first live sample produced valid structured output, but proposed actions such as paging staff, allocating a room and checking lab availability. This was a medium-severity issue with high priority for correct demonstrations. No such action was executed.

The prompt did not clearly describe the available tools. We changed it to describe supported responsibilities, limit ActionTool to appointment search and keep doctor approval explicit. We then repeated the same three inputs. All three returned supported tasks in that retest; 40 deterministic tests also passed after the change.

This shows why checking JSON format alone is not enough. Model wording can still vary, so backend tool restrictions and approval checks remain necessary. DEFECT_REPORT.md contains the before/after evidence and reproduction details. No additional defects were invented to fill the report.

7. Reliability, accessibility and remaining limits

Backend tests confirmed that only one simultaneous assignment claims a nurse, completion releases that nurse, and the waiting case can then be assigned. Another test checked recovery from an arranged interrupted state. Flutter tests confirmed that expired sessions and failed telemetry/completion do not appear successful. These were not real device-network or process-crash experiments.

Four axe-core form checks found no definite violations. However, multiple-label checks on registration, consultation and dispatch need manual review. Color contrast was disabled in jsdom. Keyboard use, screen-reader behavior and real-browser/device compatibility remain unverified in this evaluation.

Other known product limits include the shared doctor review queue, foreground location tracking, estimated straight-line travel time and simulated SMS. The tests do not remove those limits. Live AI evidence covers only a small planner sample, not every agent or clinical accuracy.

8. Conclusion and submission status

The recorded tests support the selected functional, integration, performance, security and reliability checks. One real planner defect was corrected and retested. The results do not justify calling the system production-ready or claiming every possible case is covered.

Before submission, the team must confirm role mapping and real contributions, review the remaining manual checks, record the final repository/commit links and complete the module's AI declaration. Each member must personally run and explain a meaningful test during the viva. See SUBMISSION_CHECKLIST.md. INDIVIDUAL_TESTING_GUIDE.md gives each member a practical starting point; CONTRIBUTION_RECORD.md records their actual work.

9. Supporting files and AI assistance

Submit this PDF with TEST_PLAN.md, TEST_CASES.csv, DEFECT_REPORT.md, EXECUTION_SUMMARY.md, EVIDENCE_INDEX.md, the linked evidence folders and test source/scripts. CLIENT_INTEGRATION.md and the AI, performance, security and non-functional evaluation files provide detailed rerun instructions.

AI assistance was used for test ideas, test code, debugging, evidence interpretation and documentation, including this report. Students must verify and understand the work and describe their own actual contributions. The module's exact CLEAR declaration format has not yet been supplied; this statement does not replace it.